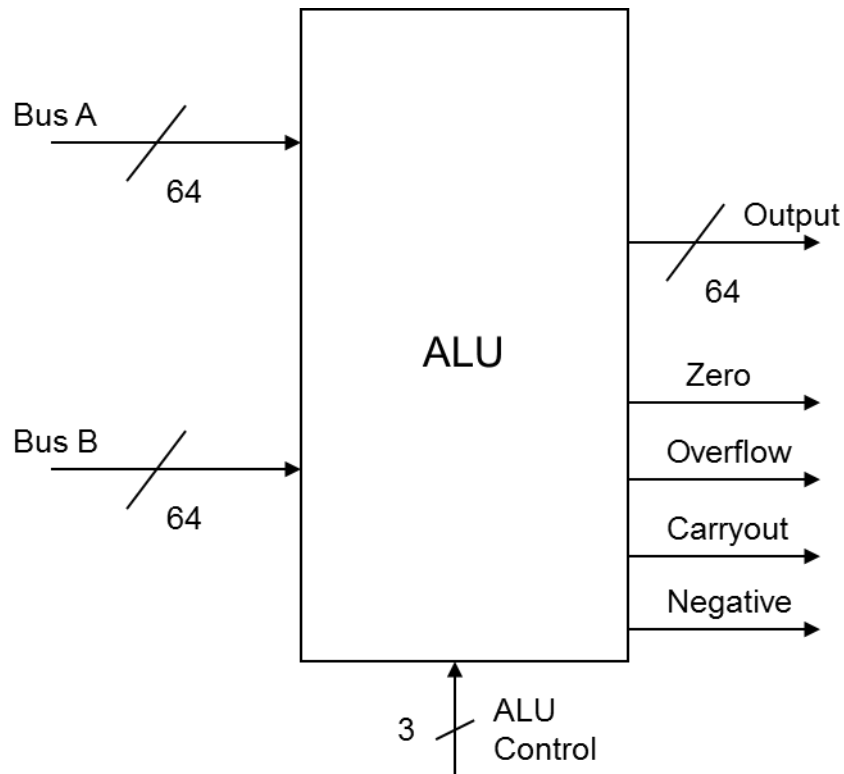


ARM ALU

Introduction: For this project you are to design a simple 64-bit ARM ALU. Examples of this type of architecture is shown in chapter 4 of your textbook. The overall block diagram of your design will look like the figure below.



The ALU has 8 ports. These ports are the two input ports A and B, the output port, ALU control, zero detect output, overflow detect output, negative detect output, and the carryout output. Like with lab #1, there is a stimulus file provided for you (though it is woefully inadequate..). That file includes comments that state the meaning of each output flag, as well as the values of the ALU control signals and the corresponding function that should be computed. Remember that the turn-in format and design requirements from lab #1 apply to this lab as well.

TURN-IN

For this lab you will electronically submit and demo the functionality of your ALU to the TAs. As with the last lab, use the provided testbench skeleton "alustim.v" to test your project. Note that the testing there is NOT complete. This is deliberate – we give you enough of an example to see how to construct the tests, while having you figure out all the cases to test yourself. As before, the TAs will have another testbench to use to test your work that is VERY thorough...

Submission Formatting

We will be using an autograder for this lab, so it is very important that you format your files in a specific way. Please place all your design files in one folder. This includes all .sv files needed for your design in addition to a top-level testbench and wavefile. Name the folder your name(s) followed by the lab number. Zip the folder and then submit it on canvas. The example below shows how your files should be formatted when you submit on canvas. Failure to follow this format will result in a warning followed by a 10% deduction.

<student_name(s)>_<lab_number>.zip
design_files.sv, runlab.do

